ISM 6562 · Big Data for Business · University of South Florida

SPORTLYTICS

Athletics Professional basketball analytics platform

Team BigQuest Query

Marta Falceto Font
Kadiatou Sogodogo
Karthikeyan Atmakuru

https://github.com/KarthikeyanAtmakuru/ISM6562-Final_project_BigQuest_Query.git

AGENDA

01 Business Problem
The $780M challenge and why the existing pipeline failed

02 Architecture Diagram
Lambda architecture — 4 stages, 19 Docker services

03 HDFS + Orchestration
Data lake zones, block replication, Airflow DAGs

04 Spark
Batch transforms — 3 joins, 11 aggregations, Parquet outputs

05 Streaming
Kafka pipeline — 25fps telemetry, fatigue alerts, <5 sec

06 Business Insights
Findings across Q1–Q6 and six recommendations for the league

07 Key Takeaways
Challenges encountered and what we would do differently

The Business Problem

Why Sportlytics needed a big data pipeline

1

45-second lag

In-game dashboard delay

Cameras fire at 25 fps but data is processed in nightly batches. Coaches cannot make real-time substitution decisions during live games.

Fix: Replace batch with Kafka + Spark Streaming

2

38% recall

Injury model accuracy

$780M in preventable player injuries last season. Incomplete features and low-quality predictions miss 62% of injuries before they happen.

Fix: Rich feature engineering from 5 joined datasets

3

Batch failures

During playoff volume spikes

Single server ingests and processes simultaneously. Resource contention causes nightly report failures exactly when reliability matters most.

Fix: Separate storage (HDFS) from compute (Spark)

Root cause: batch-only architecture cannot support real-time decisions at 25 fps across 30 NBA teams

Architecture Diagram

Lambda architecture: batch layer + speed layer | 4 stages | 19 Docker services

Karthikeyan

Stage 1

HDFS

Data Lake

- **Raw → Processed**
- **→ Analytics**

Marta

Stage 2

Spark

Batch Transform

- **3 joins · 11 aggs**
- **14 Parquet files**

Kadiatou

Stage 3

Kafka

Real-Time Stream

- **25fps telemetry**
- **60s fatigue alerts**

Karthikeyan

Stage 4

Airflow

Orchestration

- **Batch DAG · 6AM**
- **Stream monitor**

Airflow orchestrates and monitors all stages

Stage 1: HDFS Data Lake

localhost:9870 | 1 NameNode | 3 DataNodes | Replication factor 2

/sportlytics/raw/

5 files | 158 MB | Replic=2

player-tracking.csv
game-stats.json
injury-reports.csv
training-sessions.json
team-schedules.csv

/sportlytics/processed/

3 Parquet files (Stage 2)

tracking_with_stats.parquet
training_with_schedule.parquet
tracking_stats_schedule.parquet

/sportlytics/analytics/

4 Parquet files + alerts (Stage 2+3)

rolling_workload.parquet
back_to_back_analysis.parquet
travel_rest_analysis.parquet
training_load_analysis.parquet

5	2.0x	3	0
Raw files loaded	Block replication	Live DataNodes	Missing blocks

Stage 2 — Spark Batch Transformation

Apache Spark 3.5.0

Marta Falceto Font

RAW ZONE

/sportlytics/raw/

5 files · 158 MB

CSV & JSON

→

SPARK JOINS

tracking + stats + schedule

3 enriched datasets

-> /processed/

→

ANALYTICS

/sportlytics/analytics/

11 aggregations

14 Parquet outputs

750K

tracking events processed

14

Parquet files written to HDFS

11

Aggregations

154K

correctly joined training rows

Stage 3 — Real-Time Streaming Pipeline

The Problem: Coaches were making substitution decisions on data 45 seconds old.

BUSINESS NEED

Teams need alerts during live games

HOW WE SOLVED IT

Kafka streams player telemetry at 25fps. One event per tracking frame published to the realtime_player_telemetry topic with 3 partitions for parallel consumption.

BUSINESS NEED

Alerts must be personalized per player

HOW WE SOLVED IT

Thresholds derived from each player's own historical data in Stage 2 rolling_workload.parquet, not population averages. HR threshold: 95% of personal max.

BUSINESS NEED

Collective Bargaining Agreement compliance requires workload tracking

HOW WE SOLVED IT

Every fatigue alert is persisted to HDFS at streaming_alerts.parquet and monitored by the Stage 4 Airflow streaming health DAG.

Stage 3 — What the Pipeline Delivered

296,177	20,179	89%	450
Windows processed	Fatigue alerts fired	Latency reduction	Players monitored

Heart Rate Alerts

PLR-0265 (TEAM-04) and PLR-0296 (TEAM-13) consistently hit 220 BPM, the physiological ceiling, across multiple games. TEAM-07 led the league with triggers, revealing players routinely pushed to their absolute cardiovascular limit.

→ These players are substitution candidates the moment an alert fires, not after the quarter ends.

Distance Alerts

TEAM-10 led the league with 756 distance alerts and 866 total fatigue alerts across 15 monitored players. Distance alerts were far more common than heart rate alerts overall, revealing that per-minute workload, not peak intensity, is the dominant fatigue driver.

→ TEAM-10's rotation strategy needs adjustment, their players are being systematically over-extended.

Before: 45-second lag — coaches acted on stale data → After: <5 seconds — actionable within the same defensive possession

Stage 4: Airflow Orchestration

localhost:8085 | admin / admin | 2 DAGs

Batch pipeline DAG

sportlytics_batch_pipeline | Daily 6AM UTC

1. ingest_game_data_to_hdfs
2. data_quality_gate
3. check_impossible_values
4. check_tracking_completeness
5. run_spark_joins
6. run_spark_aggregations
7. update_rolling_workload
8. refresh_injury_risk_scores
9. verify_outputs
10. log_pipeline_success

Retries=2 | SLA=2hrs | email_on_failure=True | Idempotent

Streaming monitor DAG

sportlytics_streaming_monitor | Every 30min | 6PM to midnight

`check_kafka_broker`

Kafka broker at kafka:9092 reachable

`check_kafka_topic`

realtime_player_telemetry topic exists

`check_streaming_checkpoint`

HDFS checkpoint directory exists

`check_alerts_output`

Fatigue alerts written (soft failure)

`report_streaming_status`

Log full health summary + timestamp

Game hours only | Retries=1 | SLA=15min

docker compose up -d >> 01_load_hdfs.sh >> Airflow at localhost:8085 >> Jupyter at localhost:8888

Business Insights — What the Data Revealed

Q1 Injury Risk Prediction

Near-zero Pearson correlations across all workload features confirm injury risk is nonlinear — raw volume alone cannot predict injuries.

→ A Random Forest model trained on the 144,583-record labeled dataset built by Stage 2 is the next step toward replacing the 38% recall model.

Q2 Fatigue-Performance Correlation

Shooting percentage remained flat (49.77%–50.61%) across all four workload quartiles — no significant decline threshold found at the league level.

→ League-wide averages mask individual breakpoints. Personalized thresholds will reveal player-specific fatigue patterns that aggregate analysis hides.

Q3 Back-to-Back Impact

High-usage players drop from +0.021 to -0.074 plus-minus on back-to-back nights (-0.095 drop). Low-usage players actually improve (+0.098), likely from increased minutes.

→ Blanket rest policies are unnecessary. Targeted rest for high-usage players on consecutive nights is sufficient and more efficient.

Q4 Travel and Rest Interaction

Travel distance had near-zero correlation with performance (r=0.005). Games with 4+ rest days yielded the best plus-minus (+0.158) regardless of trip distance.

→ Teams should prioritize schedule density over travel logistics when planning road trips.

Business Insights

Q5

Training Load Optimization

High-intensity weeks were associated with better performance (plus-minus +0.509 vs +0.176), suggesting they coincide with peak fitness, not fatigue.

Optimal ratio: 1–2 practice sessions per recovery session yielded the best plus-minus (+0.474) and shooting % (50.83%). Exceeding 2:1 erodes shooting efficiency.

→ Target 1.5 practice sessions per recovery session per week.

Q6

Real-Time Exertion Monitoring

296,177 windows · 20,179 alerts · <5 sec latency

TEAM-10 had the most fatigue alerts (866), TEAM-07 the most HR alerts (223). Heart rate thresholds ranged from 186–209 BPM across all 450 players, all personalized from their own historical data.

→ 89% latency reduction — alerts reach coaches in the same possession.

Recommendations for the League

01	Move to personalized thresholds because league averages can hide individual breakpoints
02	Rest high-usage players on back-to-back nights — targeted, not blanket policies
03	Target 1.5:1 practice-to-recovery ratio — flag breaches using training load output
04	Prioritize rest days over travel logistics — schedule density drives performance
05	Use fatigue alerts as live substitution signals — not post-game summaries
06	Build a Random Forest injury model — 144K labeled records from Stage 2 are ready

Key Takeaways

What worked · What was hard · What we'd do differently

✓ What Worked

- Lambda Architecture separation of batch vs. streaming concerns was the right call
- Three quality gates made failures self-diagnosing — task name alone pinpointed the layer
- Idempotent Spark overwrite mode let us re-run any stage safely without duplicates
- team_id partitioning reduced per-query I/O by 30× — immediate, measurable payoff

⚠ What Was Hard

- Spark version mismatch (4.1.1 vs 3.5.0) caused cryptic connection failures — pin images early
- training_sessions join exploded 150K → 436K rows without team_id key; join design needs upfront planning
- WSL2 RAM defaults crushed Spark jobs; .wslconfig tuning should be Day 1, not a fire-drill
- Running producer + streaming notebook simultaneously exhausted all 4 worker cores

↗ Do Differently

- Build a Random Forest on the 144K labeled injury records — replace the 38% recall baseline
- Add date partitioning alongside team_id before data volume grows past 300 teams
- Migrate single Kafka broker to a multi-broker cluster for replication before production
- Move to personalized fatigue thresholds league-wide — aggregate metrics hide individual breakpoints

Thank You

Team BigQuery Quest

Marta Falceto Font · Kadiatou Sogodogo · Karthikeyan Atmakuru